

Inheritance

Class Hierarchies



SoftUni Team
Technical Trainers



**Software
University**



**SoftUni
Foundation**



Software University

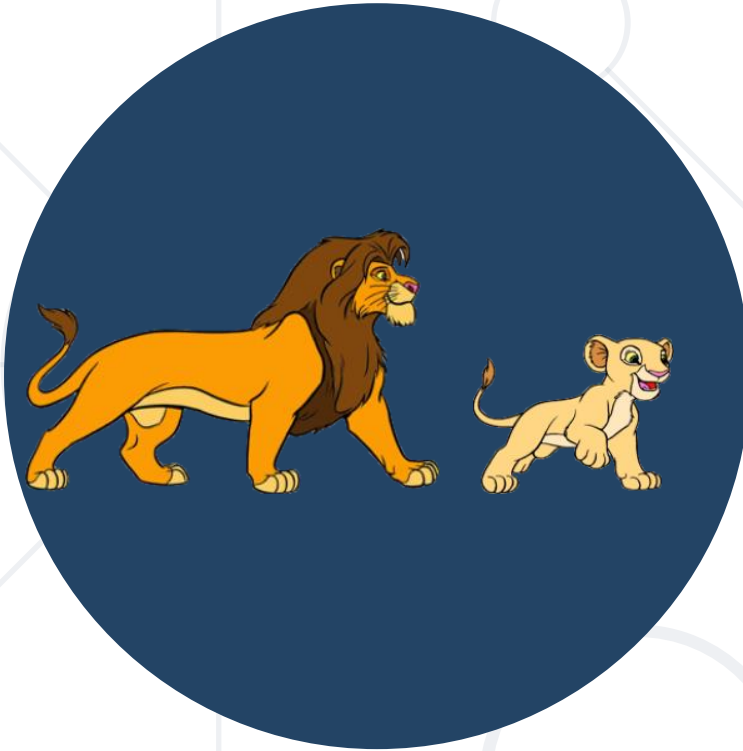
<http://softuni.bg>

1. Inheritance
2. Class Hierarchies
3. Inheritance in C#
4. Accessing Members of the Base Class
5. When to Use Inheritance
6. Composition



sli.do

#fund-csharp

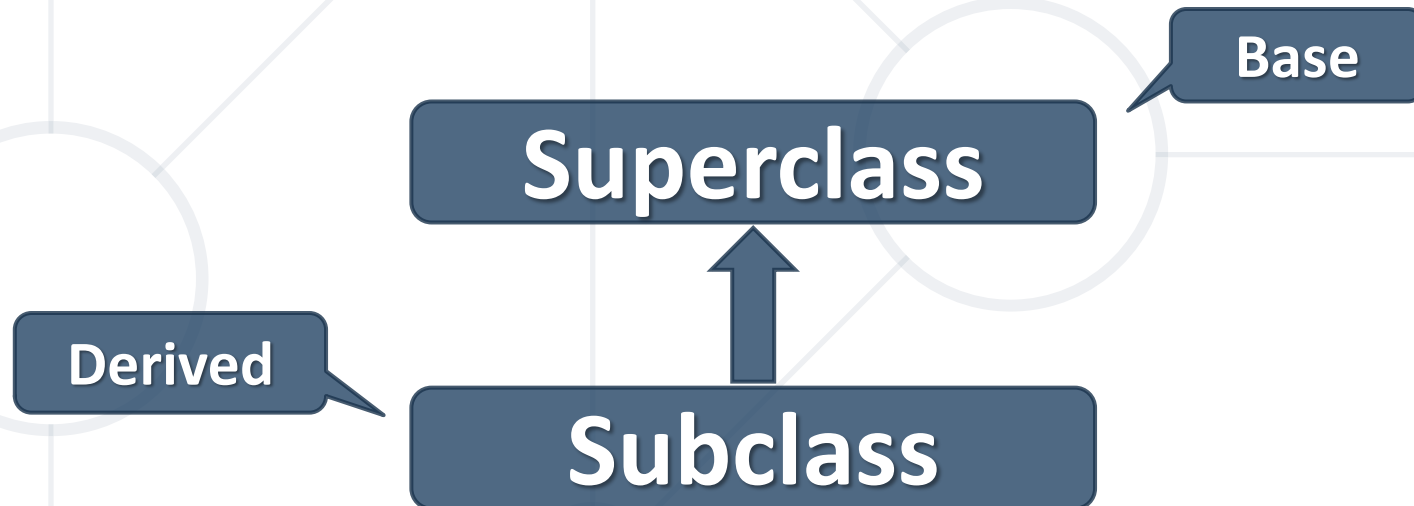


Inheritance

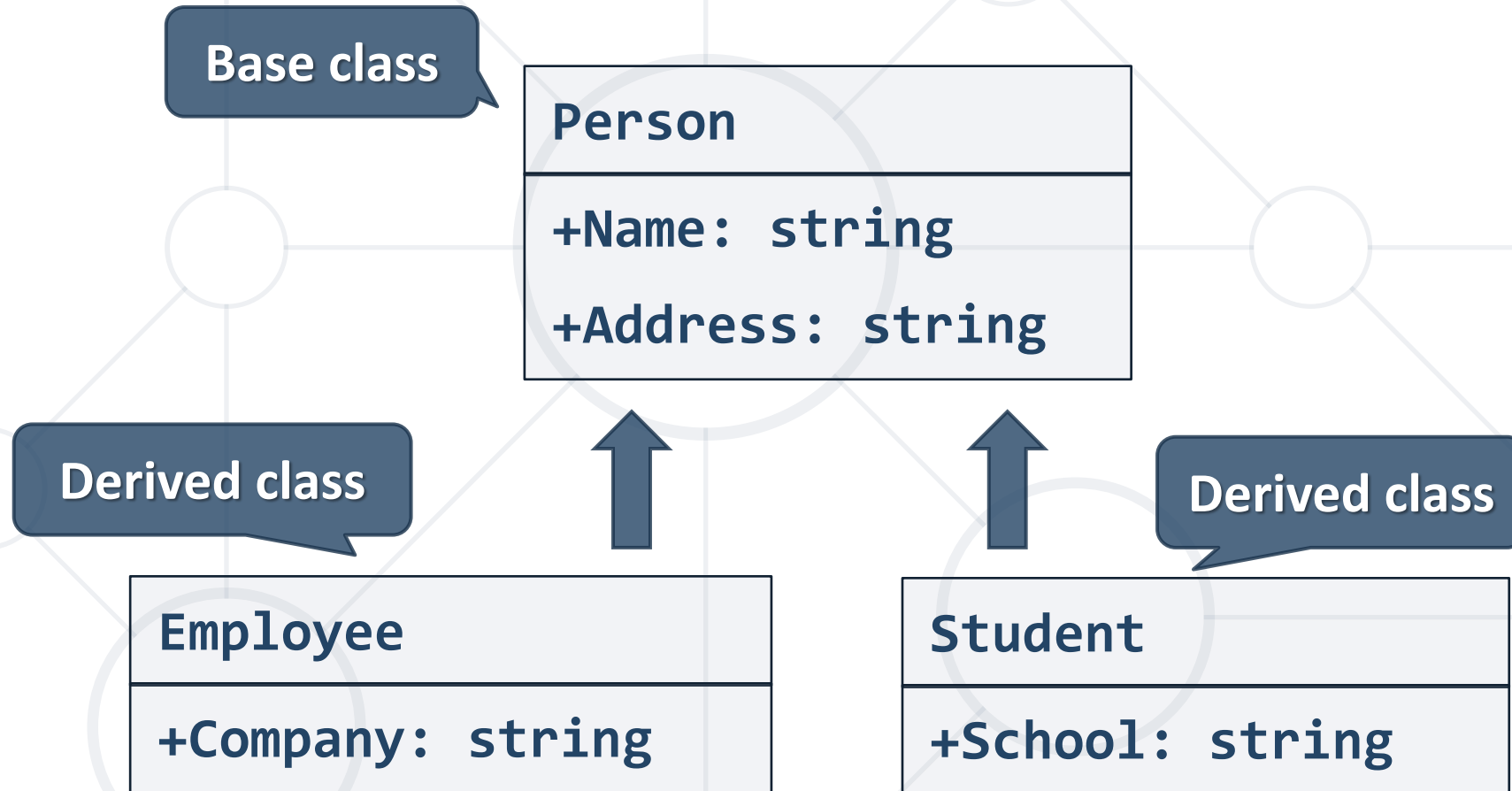
Extending Classes

Inheritance

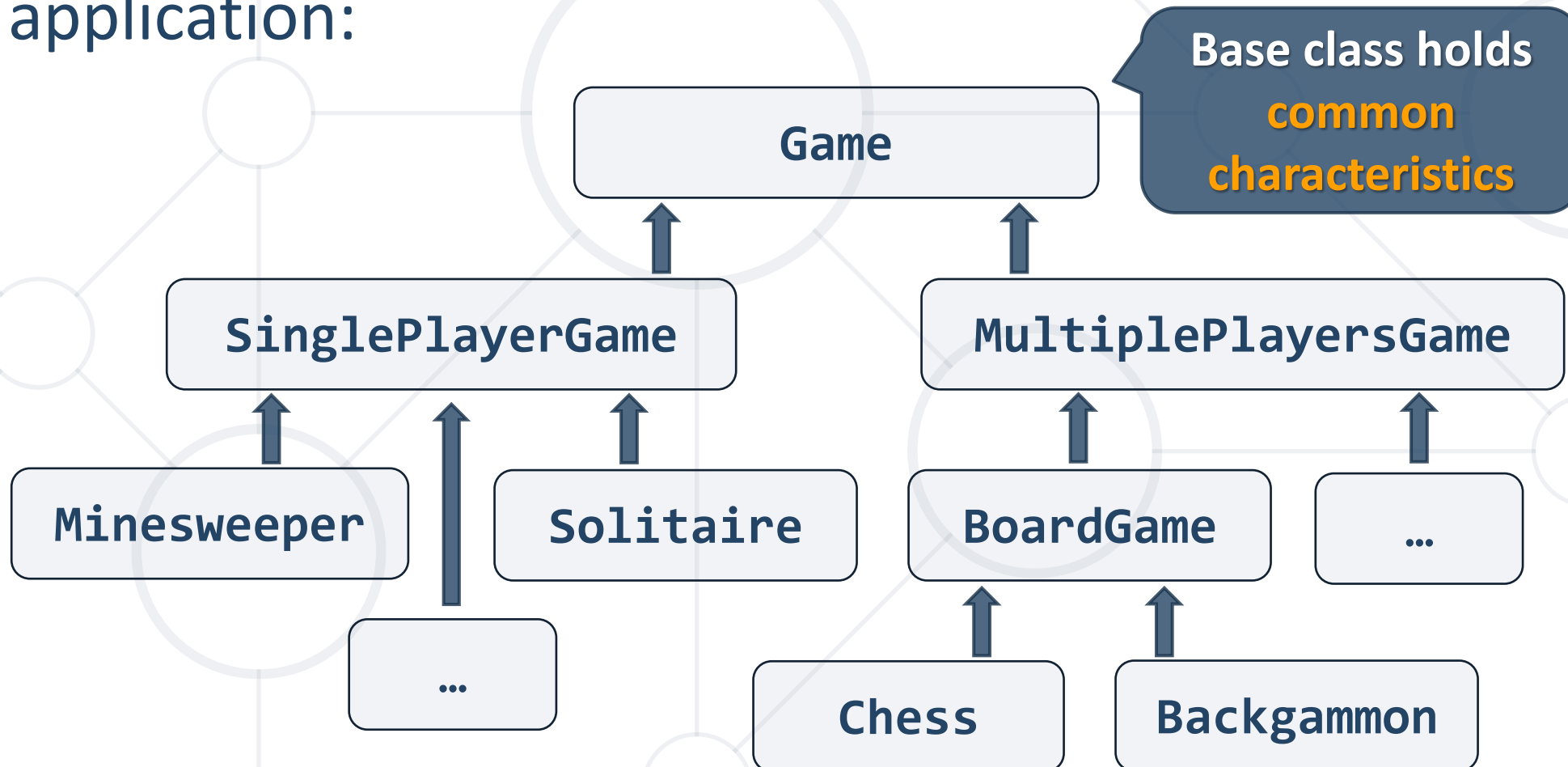
- **Superclass** - Parent class, Base Class
 - The class giving its **members** to its **child class**
- **Subclass** - **Child** class, **Derived** class
 - The class taking members from its base class



Inheritance – Example

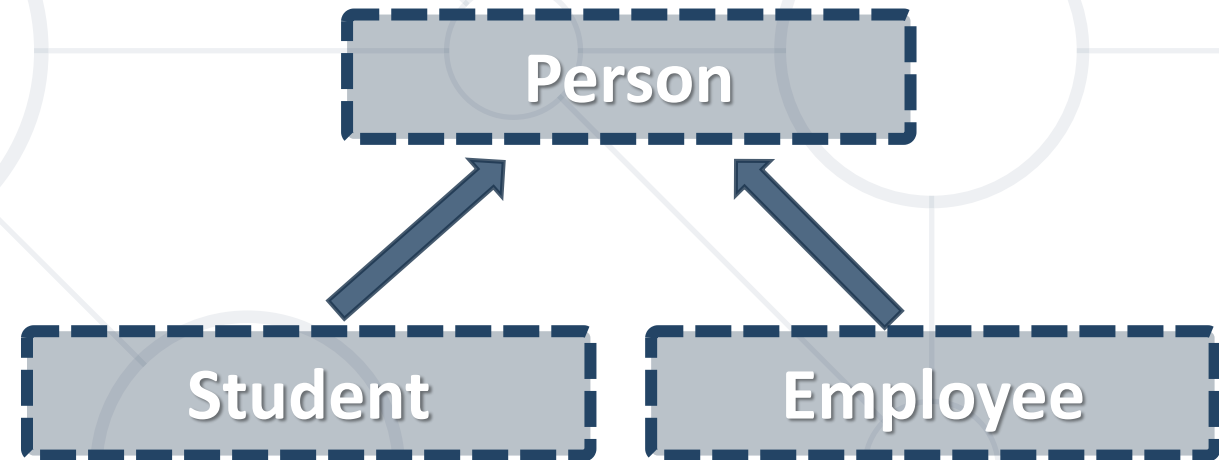


- **Inheritance** leads to **hierarchies** of classes and/or interfaces in an application:



- In C# inheritance is defined by the **:** operator

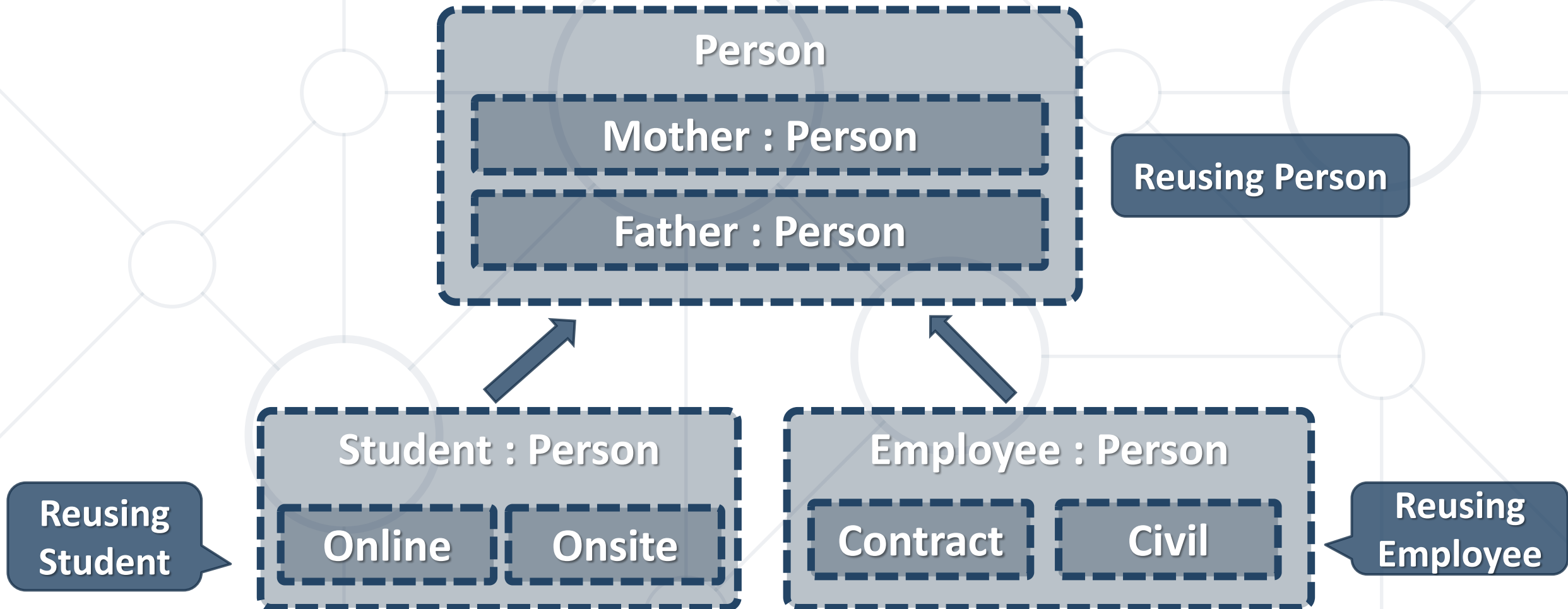
```
class Person { ... }  
  
class Student : Person { ... }  
class Employee : Person { ... }
```



Student : Person

Inheritance - Derived Class

- Derived classes **take all members** from base classes



- You can access inherited members as usual

```
class Person { public void Sleep() { ... } }  
class Student : Person { ... }  
class Employee : Person { ... }
```

```
Student student = new Student();  
student.Sleep();  
Employee employee = new Employee();  
employee.Sleep();
```

- Constructors are **not inherited**
- Constructors **can be reused** by the child classes

```
class Student : Person {  
    private School school;  
    public Student(String name, School school)  
        :base(name)  
    {  
        this.school = school;  
    }  
}
```

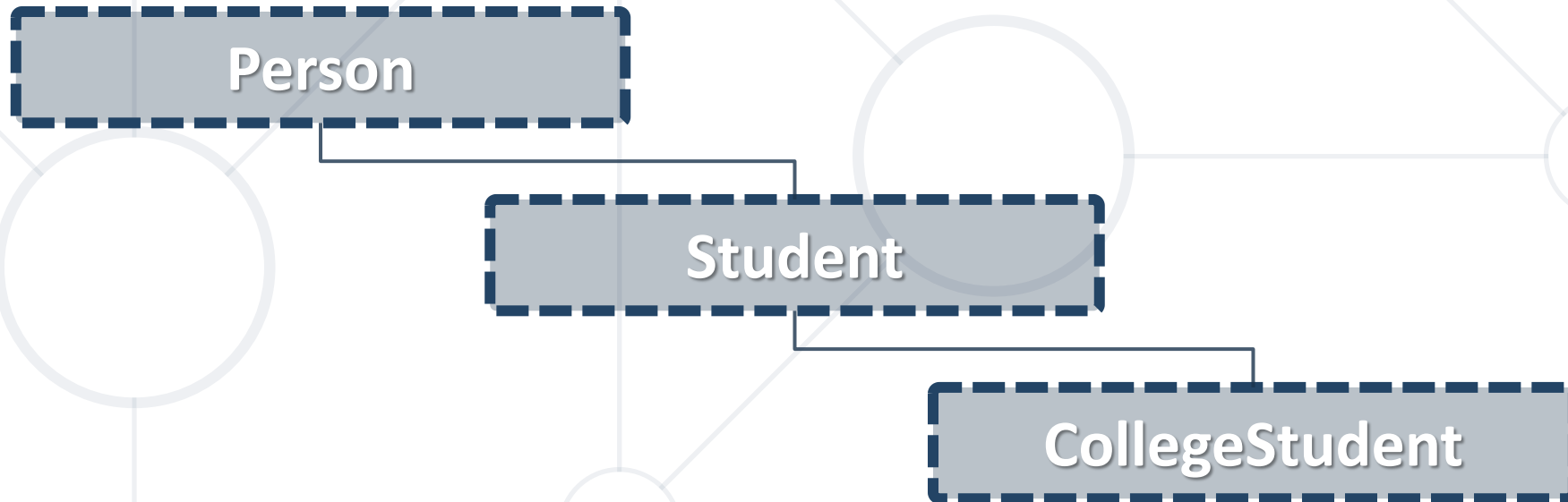
Thinking About Inheritance - Extends

- Derived class instance **contains** instance of its base class



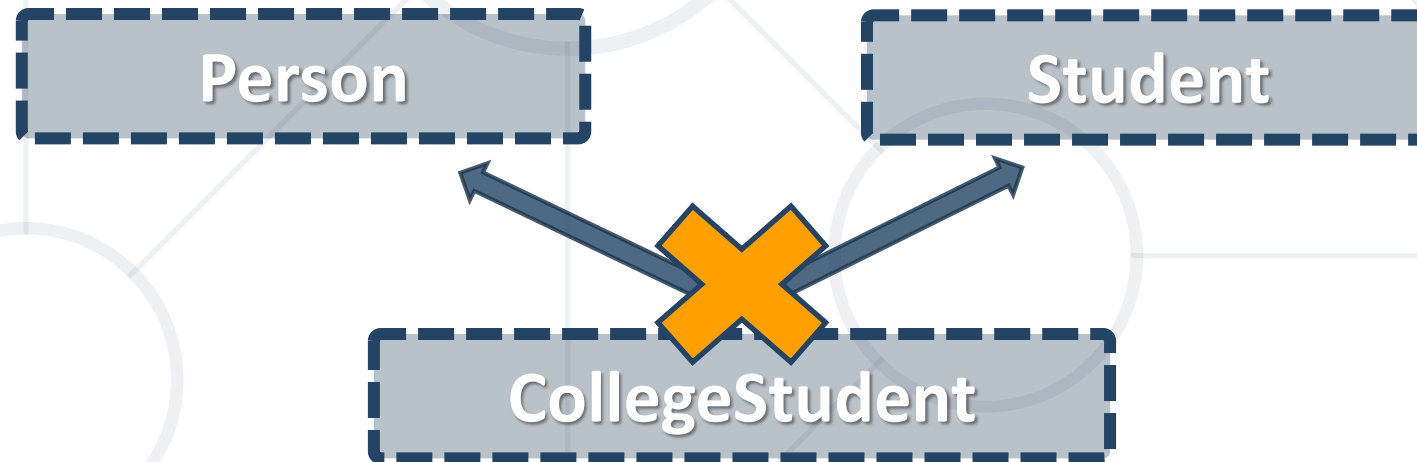
- Inheritance has a **transitive relation**

```
class Person { ... }  
class Student : Person { ... }  
class CollegeStudent : Student { ... }
```



Multiple Inheritance

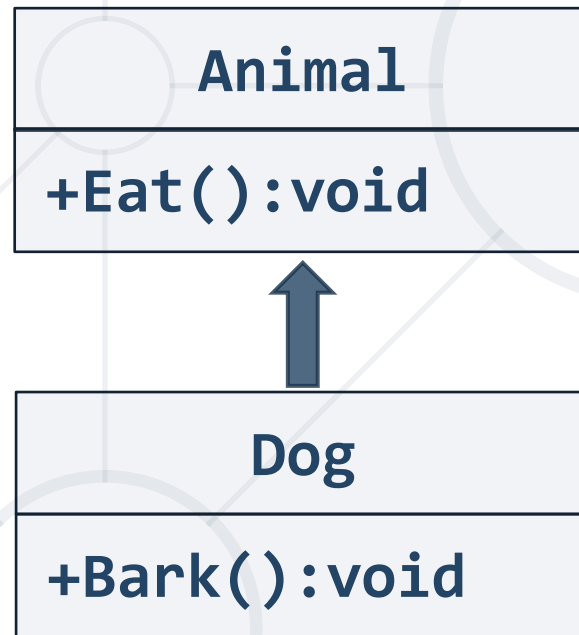
- In C# there is no **multiple** inheritance
- Only **multiple interfaces can be implemented**



- Use the **base** keyword

```
class Person { ... }  
class Employee : Person  
{  
    public void Fire(string reasons)  
    {  
        Console.WriteLine(  
            $"{{base.name}} got fired because of {{reasons}}");  
    }  
}
```

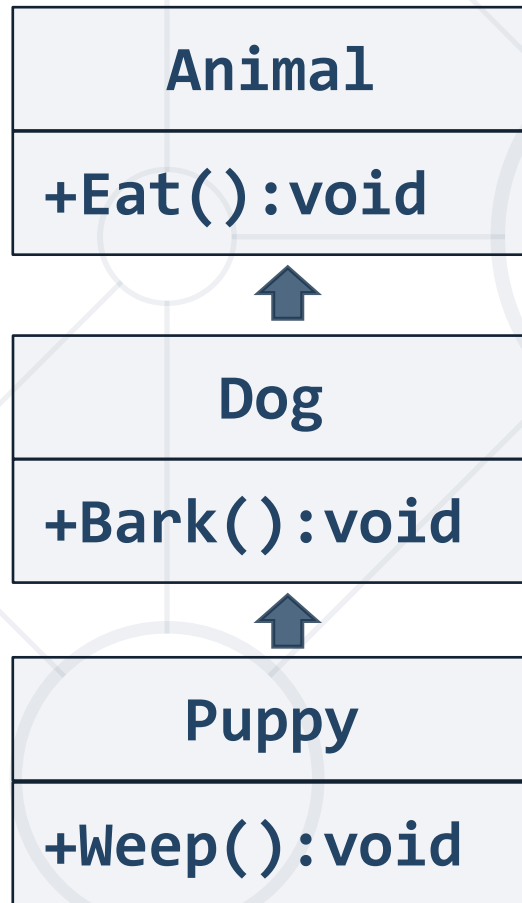
Problem: Single Inheritance



```
Dog dog = new Dog();  
dog.Eat();  
dog.Bark();
```

Check your solution here: <https://judge.softuni.bg/Contests/679/Inheritance-Lab>

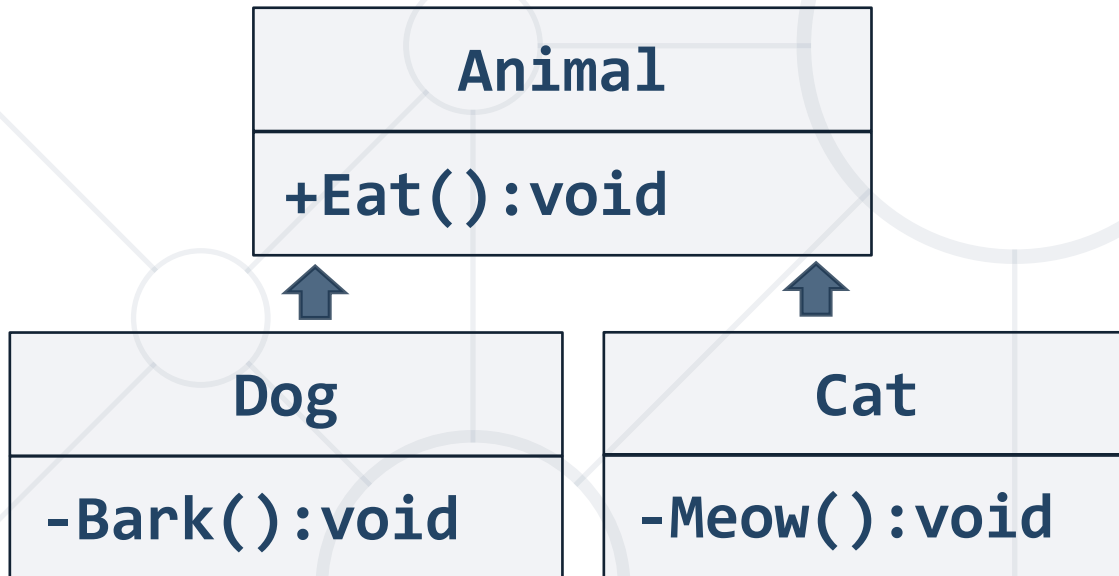
Problem: Multilevel Inheritance



```
Puppy puppy = new Puppy();
puppy.Eat();
puppy.Bark();
puppy.Weep();
```

Check your solution here: <https://judge.softuni.bg/Contests/679/Inheritance-Lab>

Problem: Hierarchical Inheritance



```
Dog dog = new Dog();
dog.Eat();
dog.Bark();

Cat cat = new Cat();
cat.Eat();
cat.Meow();
```

Check your solution here: <https://judge.softuni.bg/Contests/679/Inheritance-Lab>



Reusing Classes

Reusing Code at Class Level

- Derived classes **can access all public** and **protected** members
- They can access **internal** members **if in same assembly**
- **Private** fields are **not inherited** in subclasses

```
class Person {  
    private string id;  
    string name;  
    protected string address;  
    public void Sleep();  
}
```

Can be accessed through
other methods

- Derived classes **can hide** superclass variables

```
class Person { protected int weight; }
```

```
class Patient : Person
```

```
{
```

Hides **int weight**

```
    protected float weight;
```

```
    public void Method()
```

```
{
```

```
    double weight = 0.5d;
```

Hides **float weight**

```
}
```

```
}
```

- Use **base** and **this** to specify member access

```
class Patient : Person
```

```
{
```

```
    protected float weight;
```

```
    public void Method() {
```

```
        double weight = 0.5d;
```

```
        this.weight = 0.6f;
```

```
        base.weight = 1;
```

```
    }
```

```
}
```

Local variable

Base class member

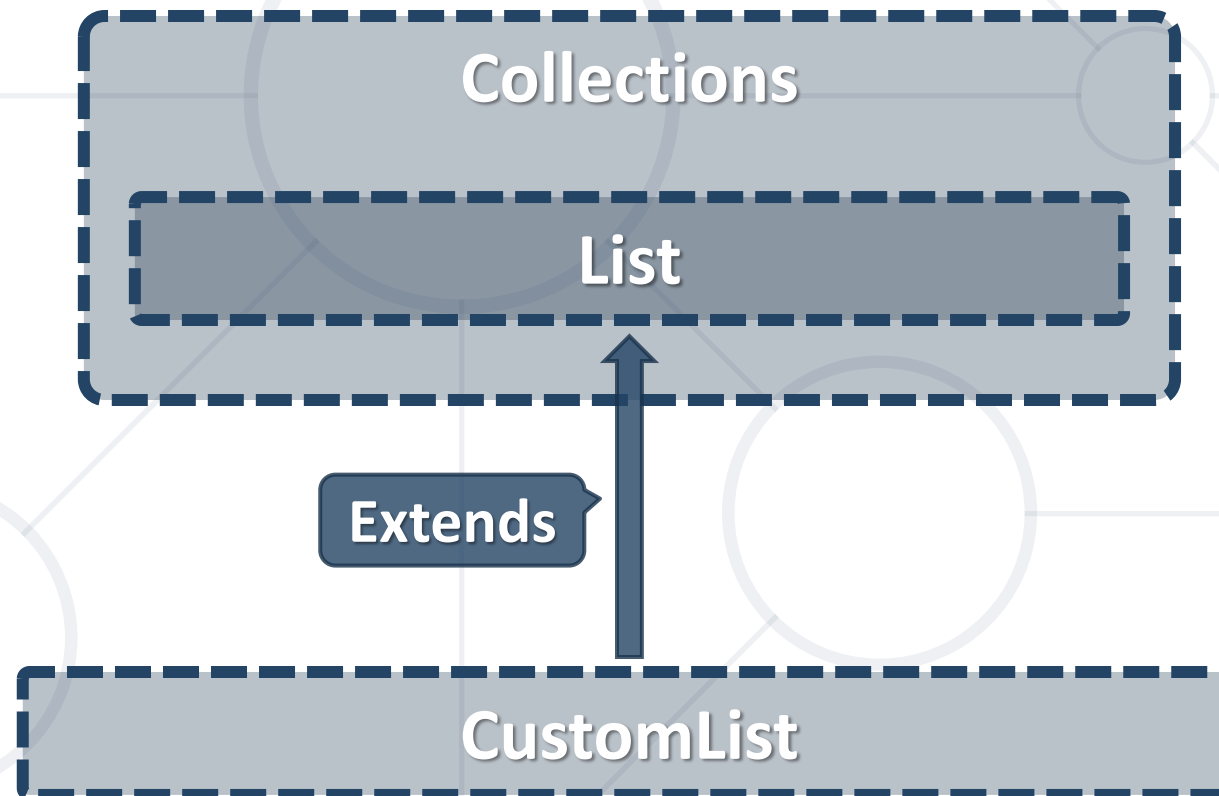
Instance member

- **virtual** – defines a method that **can be overridden**

```
public class Animal
{
    public virtual void Eat() { ... }
}
```

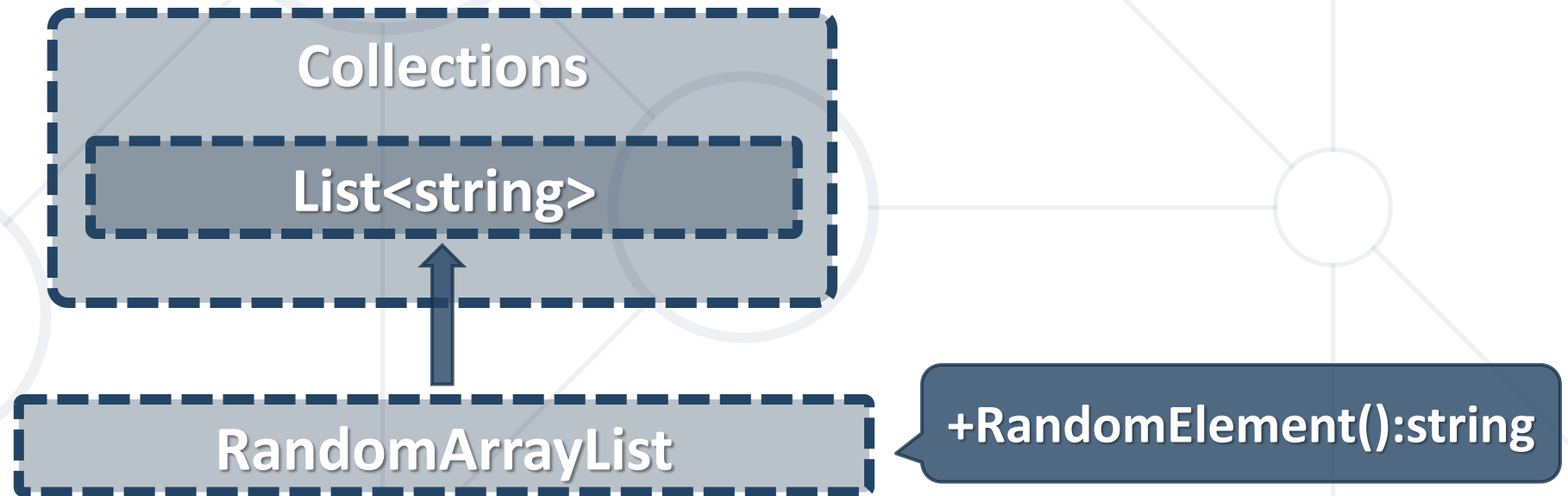
```
public class Dog : Animal
{
    public override void Eat() {}
}
```

- We can **extend a class** that we **can't otherwise change**



Problem: Random List

- Create an array list that has
 - All functionality of a **List<string>**
 - Function that returns and removes a random element



Solution: Random Array List

```
public class RandomList : List<string> {  
    private Random rnd; //TODO: Add constructor  
    public string RemoveRandomElement()  
    {  
        int index = rnd.Next(0, data.Count - 1);  
        string str = data[index];  
        data.RemoveAt(index);  
        return str;  
    }  
}
```



Types of Class Reuse

Extension, Composition, Delegation

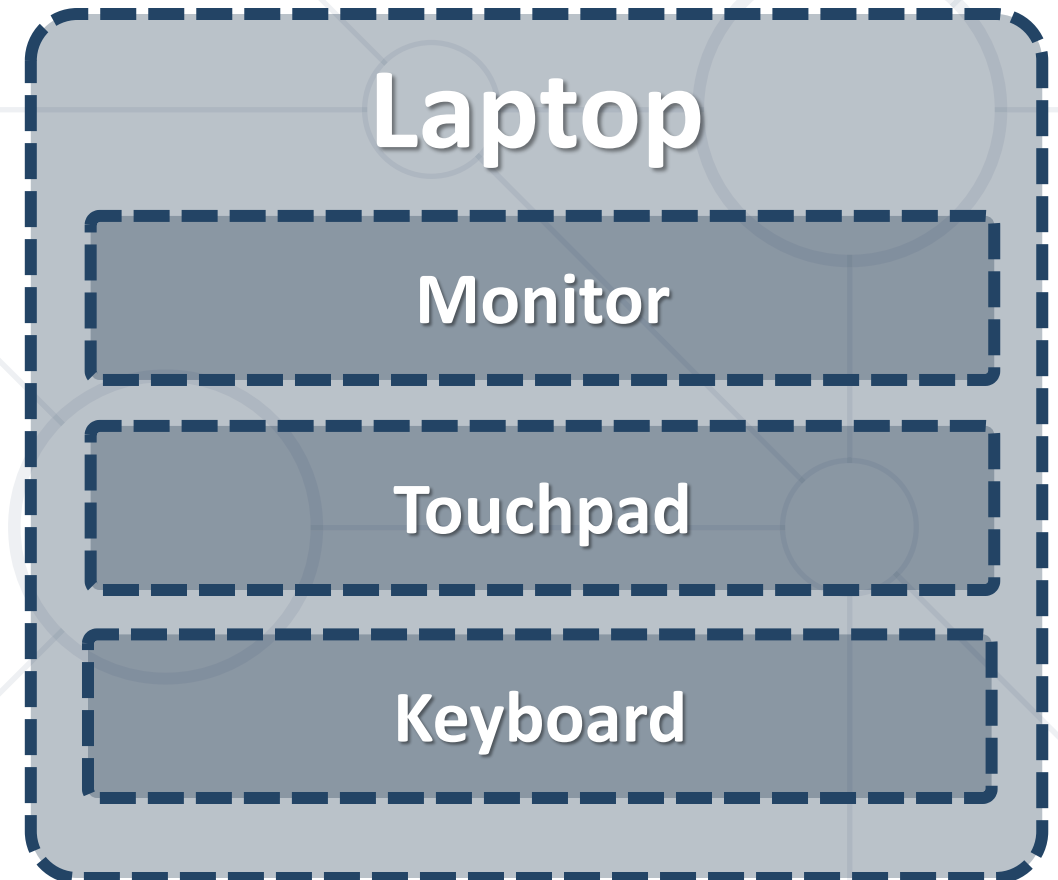
- **Duplicate code** is error prone
- **Reuse classes** through **extension**
- Sometimes the only way



- Using classes to define classes

```
class Laptop {  
    Monitor monitor;  
    Touchpad touchpad;  
    Keyboard keyboard;  
    ...  
}
```

Reusing
classes



Delegation

```
class Laptop
{
    Monitor monitor;
    void IncrBrightness()
        monitor.Brighten();

    void DecrBrightness()
        monitor.Dim();
}
```



Problem: Stack of Strings

- Create a simple Stack class which can store only strings

StackOfStrings
-data: List<string>
+Push(string element): void +Pop(): string +Peek(): string +IsEmpty(): boolean



Solution: Stack of Strings

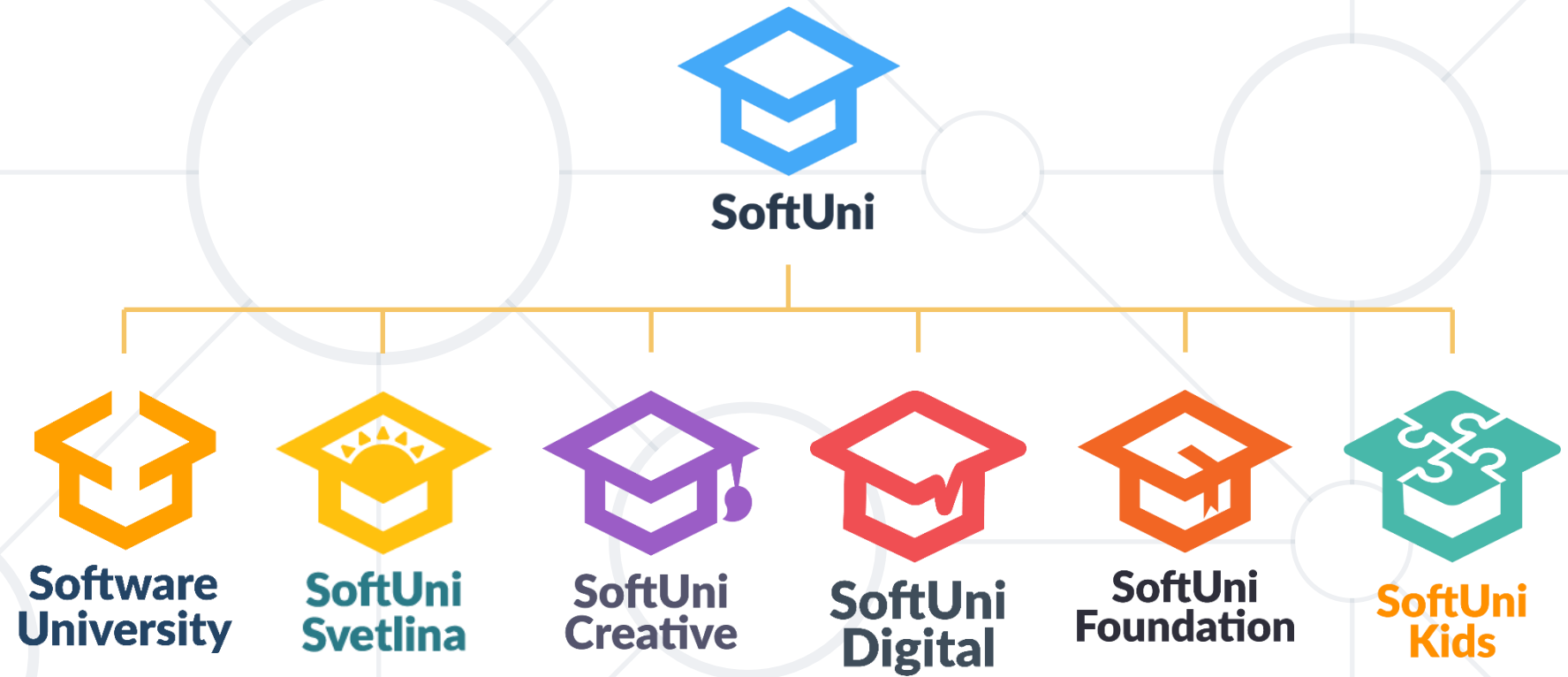
```
public class StackOfStrings {  
    private List<string> data;  
    public void Push(string element)  
    { this.data.Add(element); }  
    public string Pop() {  
        var element = this.data.Last();  
        this.data.Remove(element);  
        return element;  
    }  
}
```

TODO: Validate if list is
not empty

- Inheritance is a powerful tool for **code reuse**
- **Subclass inherits** members from **Superclass** and can **override** methods
- Look for classes with the **same role**
- Look for **IS-A** and **IS-A-SUBSTITUTE**
- Consider **Composition** and **Delegation**



Questions?



SoftUni Diamond Partners



XSsoftware



SBTech



telenor



SoftwareGroup
doing it right

NETPEAK



SmartIT



Postbank

Решения за твоето утре

**SUPER
HOSTING
.BG**

INDEAVR

Serving the high achievers



INFRAGISTICS®

LIEBHERR



æternity



codexio

SoftUni Organizational Partners



Trainings @ Software University (SoftUni)

- Software University – High-Quality Education and Employment Opportunities
 - softuni.bg
- Software University Foundation
 - <http://softuni.foundation/>
- Software University @ Facebook
 - facebook.com/SoftwareUniversity
- Software University Forums
 - forum.softuni.bg



- This course (slides, examples, demos, videos, homework, etc.) is licensed under the "Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International" license

